



SESSION 4

BUILT-IN MODULES

Learning Objectives

In this session, students will learn to:

- Describe built-in modules in Node.js
- Explain `http` module to create an HTTP server and how to use it
- Explain how to use the `url` module to parse and work with URLs
- Explain how to use the `events` module to handle asynchronous events

Node.js comes with a variety of built-in modules that are readily available for use without any additional installations. These modules provide essential functionality to Node.js applications and are included as a part of the core library of Node.js. The built-in modules serve as a foundation for building a wide range of applications, from Web servers to file systems, and much more.

Built-in modules are optimized for the Node.js runtime and so are faster than third-party modules. These modules are maintained by the Node.js team and therefore, are reliable and secure.

This session will discuss three important built-in modules: `http`, `url`, and `events`.

4.1 http Module

The `http` module in Node.js allows users to create HTTP servers and make HTTP requests. This module provides various classes and methods that help implement interactions with HTTP protocols in an organized and efficient manner.

The most commonly used classes and their methods in the `http` module are as follows:

<code>http.Server</code>	• <code>http.createServer</code> • <code>server.listen</code> • <code>server.close</code>
<code>http.ClientRequest</code>	• <code>request.write</code> • <code>request.end</code>
<code>http.ServerResponse</code>	• <code>response.write</code> • <code>response.end</code> • <code>response.writeHead</code>

4.1.1 http.Server Class

The `http.Server` class is a core part of the `http` module. It represents an HTTP server that is designed to handle incoming HTTP requests and generate responses for clients.

Table 4.1 describes methods of the `http.Server` class.

Method	Description
<code>http.createServer([options,] requestListener)</code>	This method creates an HTTP server. It accepts an <code>options</code> object and a <code>requestListener</code> function. The <code>options</code> object is optional. This object includes two internal objects <code>IncomingRequest</code> and <code>ServerResponse</code> . These options object is used to specify the classes that will be used for these two internal objects. The <code>requestListener</code> function accepts two parameters, one is the request from the client and the other is the response to the request.

Method	Description
	This function is invoked for every incoming HTTP request.
<code>server.listen(port, [hostname], [backlog], [callback])</code>	This method binds and listens to the specified port with a hostname where the hostname is optional. Additionally, users can provide a <code>backlog</code> value and a <code>callback</code> function, which is executed when the server begins listening for incoming connections.
<code>server.close([callback])</code>	This method stops the server from accepting new connections. Optionally, users can provide a <code>callback</code> function that is executed when the server is closed for new connections.

Table 4.1: Methods of `http.server` Class

4.1.2 `http.ClientRequest` class

The `http.ClientRequest` class represents an outbound HTTP request. Instances of this class are used to define the specifics of an HTTP request, including the `request` method, headers, and data to be sent to the server.

Table 4.2 describes methods of the `http.ClientRequest` class.

Method	Description
<code>request.write(chunk[, encoding][, callback])</code>	This method writes data to the request body. It allows users to provide optional <code>encoding</code> and <code>callback</code> functions, which will be executed when the data has been successfully written to the request body.
<code>request.end([data][, encoding][, callback])</code>	This method signals that the request is complete and sends any data contained in the optional <code>data</code> parameter. Additionally, users can specify the <code>encoding</code> and an optional <code>callback</code> function that will be executed when the request is completed.

Table 4.2: Methods of `http.ClientRequest` Class

4.1.3 `http.ServerResponse` class

The `http.ServerResponse` class is generated automatically by an HTTP server in Node.js. It is used to send HTTP responses to clients. Instances of this class are provided as the second argument to the request callback function.

Table 4.3 shows methods of the `http.ServerResponse` class.

Method	Description
<code>response.write(chunk, [encoding], [callback])</code>	This method is used to write the response to be sent to the client. Small portions of the data to be sent are contained in the <code>chunk</code> parameter. The method is called multiple times until all the chunks of data are sent. Additionally, the method can contain the optional <code>encoding</code> parameter and the <code>callback</code> function. The <code>encoding</code> parameter specifies how data is encoded. The <code>callback</code> function is executed when the complete response data is written.
<code>response.end([data], [encoding], [callback])</code>	This method signals the end of the response. It uses the optional <code>data</code> parameter to include any remaining data to be sent as part of the response body. It can also specify the encoding used for the data and provide a <code>callback</code> function that is executed when the response is complete.
<code>response.writeHead(statusCode, [statusMessage], [headers])</code>	This method combines the setting of the status code, status message, and headers of the response into one function. These settings are used to set the HTTP status code, associated status message, and response headers, respectively, of the response.

Table 4.3: Methods of `http.ServerResponse` Class

4.2 Creating a Simple HTTP server

Consider that a user wants to create a simple HTTP server that responds with a plain text message `Welcome to Node.js HTTP Module` for `GET` requests. To do so, the user must perform these steps:

Creating the Server

1. Open a text editor, such as VS Code or Notepad.

2. To include the `http` module, type the code as:

```
const http = require('http');
```

This makes the `http` module available for use in the script.

3. To create an HTTP server using the `http.createServer` method, continue typing the code in Code Snippet 1 into the same file.

Code Snippet 1:

```
const server = http.createServer((req, res) => {  
  // Request handling code here  
});
```

In Code Snippet 1, the `req` parameter represents the incoming request and the `res` parameter represents the response of the server.

4. To handle different types of HTTP requests (`GET`, `POST`, and more) by examining the `req.method` property, in the `callback` function of the server, type the code given in Code Snippet 2.

Code Snippet 2:

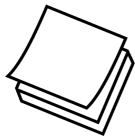
```
const server = http.createServer((req, res) => {  
  if (req.method === 'GET') {  
    // Handle GET request  
  }  
});
```

5. To set the format of the response from the server to plain text and specify the status code as `200`, in the `if` block, add the code in Code Snippet 3.

Code Snippet 3:

```
res.writeHead(200, { 'Content-Type': 'text/plain' });  
res.end('Welcome to Node.js HTTP Module \n');
```

In Code Snippet 3, the `res.writeHead` method is used to set the HTTP status code and headers and the `res.end` method is used to send the response body. The user can also use the `setHeader` method instead of the `writeHead` method to define the response type of the server.



In Node.js, HTTP headers are essential for specifying the type of response, plain text, HTML, JSON, or any other format, based on the requirements.

6. To specify the port on which the server should listen for the requests, after the `http.createServer` method, add the code in Code Snippet 4.

Code Snippet 4:

```
const port = 3000;  
server.listen(port, () => {  
  console.log(`Server is listening on port ${port}`);  
});
```

Code Snippet 4 sets up the server to listen on port 3000 and prints a message to the console when the server is running.

7. Save the file as `servercreate.js`.

Starting the Server

1. To start the server, at the command prompt, type the command as:

```
node servercreate.js
```

The command executes and displays the message, as shown in Figure 4.1 indicating that the server is running and listening on port 3000.

```
C:\NodeJS Programs>node servercreate.js  
Server is listening on port 3000
```

Figure 4.1: Starting the Server

2. To test the server, open a Web browser and navigate to the URL: <http://localhost:3000>.

The message Welcome to Node.js HTTP Module for GET requests is displayed, as shown in Figure 4.2.

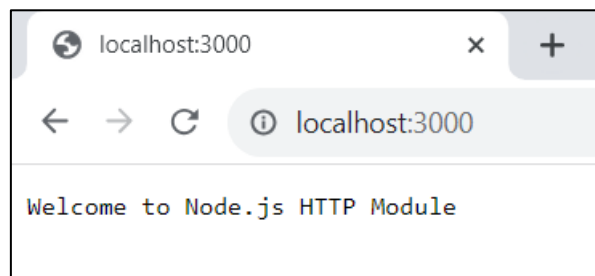


Figure 4.2: Response Message

4.3 Create an HTML Response for the HTTP Server

To get an HTML response from the HTTP server, users must include an HTTP header with the content type as text/html. To do so, perform these steps:

1. Type the code shown in Code Snippet 5 in the text editor.

Code Snippet 5:

```
const http = require('http');  
const server = http.createServer((req, res) => {  
  if (req.method === 'GET' && req.url === '/html') {  
    res.writeHead(200, { 'Content-Type': 'text/html' });  
    res.write('<b>Welcome to Node.js</b><br/>');  
    res.write('<span style="font-weight:bold; color:Red">In  
this session, we learn about HTTP module.</span>');  
    res.end();  
  } else {  
    res.writeHead(404, { 'Content-Type': 'text/plain' });  
    res.end('404 Not Found');  
  }  
});
```

```
const port = 3000;  
server.listen(port, () => {  
  console.log(`Server is listening on port ${port}`);  
});
```

In Code Snippet 5, a server is created that serves an HTML page to the clients. Different routes can be defined based on the URL of the incoming request using the `req.url` property. In Code Snippet 5, the property is set such that if the requested URL is the root (`/html`), the server will send a response with an HTML page. Otherwise, the server will send a response with a 404 Not Found message.

2. Save the file as `serverhtml.js`.
3. Open the Command Prompt window.
4. To start the server, at the Command Prompt, type the command as:

```
node serverhtml.js
```

The command executes and displays the message, as shown in Figure 4.3 indicating that the server is running and listening on port 3000.

```
C:\NodeJS Programs>node serverhtml.js  
Server is listening on port 3000
```

Figure 4.3: Starting the Server

5. To test the server, open a Web browser and navigate to the URL:
<http://localhost:3000/html>.

Since, the URL is the root (`/html`), the message `Welcome to Node.js` In this session we learn about HTTP module is displayed as shown in Figure 4.4.

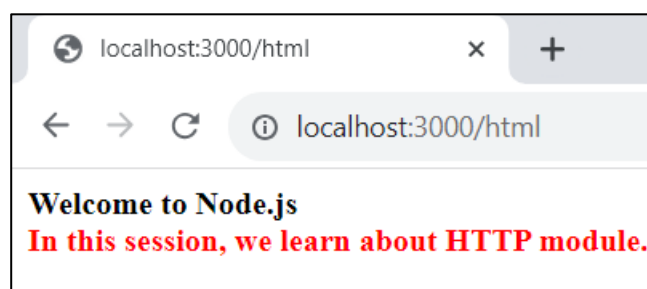


Figure 4.4: Response Message

- Now, navigate to the URL:
<http://localhost:3000/>

Since, the URL is not the root, the 404 Not Found message is displayed, as shown in Figure 4.5.

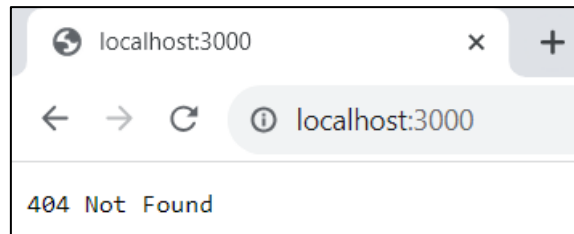


Figure 4.5: 404 Not Found Message

4.4 Create an HTTP Client Request Using the GET Method

Users can create an HTTP client request to send a GET request to a remote server. To do this, the users can use the `JSONPlaceholder` API as the remote server. This API provides a set of sample data for testing HTTP requests.

To create a GET request to the specified remote server (mentioned in hostname and path) and log the response data to the console, the user must perform these steps:

1. Type the code given in Code Snippet 6 in the text editor.

Code Snippet 6:

```
const http = require('http');
const hostname = 'jsonplaceholder.typicode.com';
const path = '/todos/61';
const options = {
  hostname: hostname,
  path: path,
  method: 'GET',
};
// Create an HTTP request
const req = http.request(options, (res) => {
  let data = '';
  res.on('data', (chunk) => {
    data += chunk;
  });
  res.on('end', () => {
    console.log(data);
  });
});
```

```
});  
});  
req.on('error', (error) => {  
    console.error(error);  
});  
// Send the GET request  
req.end();
```

Code Snippet 6 defines an `options` object that specifies the details of the HTTP request, including `hostname`, `path`, and `method`.

The `http.request(options, callback)` method is used to create an HTTP request where the `options` object contains the request details and the `callback` function will be executed when a response is received.

The response callback function listens for data chunks using the `res.on('data', ...)` event and accumulates the data.

When the response is complete, the `end` event is raised. At this point, the received data is sent to the console. The `error` event is raised on the request object to handle any errors that might occur.

2. Save the file as `requestget.js`.
3. To make a `GET` request to the specified remote server, in the Command Prompt window, at the command prompt, execute the command as:

```
node requestget.js
```

The command executes and displays the response data from the remote server, as shown in Figure 4.6.

```
C:\NodeJS Programs>node requestget.js  
{  
  "userId": 4,  
  "id": 61,  
  "title": "odit optio omnis qui sunt",  
  "completed": true  
}
```

Figure 4.6: Displays the Response Data

To summarize, developers can use the `http` module in their backend application to accept and present data through Web services.

For example, when developing a flight reservation system, the developers can use the `http` request object to:

- Send `GET` request to a route in the application for searching flights with search parameters such as origin, destination, and dates as part of the query string.
- Send `POST` requests to save the details of a specific flight booking and request an external gateway to receive payment for the flight booking.
- Send `GET` request to retrieve reservation details of a specific flight booking.
- Send `POST` request to cancel a specific flight booking.

Similarly, developers can use the `http` response object to:

- Receive a JSON response containing the flight details matching the search from the server.
- Receive a JSON response confirming the success or failure of the flight booking after interaction with external gateways for payment.
- Receive a JSON response with reservation details of a specific flight booking.
- Receive a JSON response confirming the success or failure of cancellation of a specific flight booking.

4.5 URL Module

The Uniform Resource Locator (URL) module in Node.js provides utilities for parsing and working with URLs. It allows users to:



4.5.1 Parsing the URL

The `URL` method is used to parse a URL string to create a new URL object. This method takes an input, which can be an absolute URL, (`https://www.sample.com/Sample1`) or a relative URL (`/Sample1`) with the base (`https://www.sample.com`). The URL object contains various components of the URL, such as protocol, hostname, port, pathname, and search parameters.

For example, consider that the user wants to parse the URL string: `https://www.jsonplaceholder.typicode.com/todos/61`

To do so, the user must perform these steps:

1. Type the code in Code Snippet 7 in the text editor:

Code Snippet 7:

```
const parsedUrl = new URL  
( 'https://www.jsonplaceholder.typicode.com/todos/61' );  
console.log(parsedUrl);
```

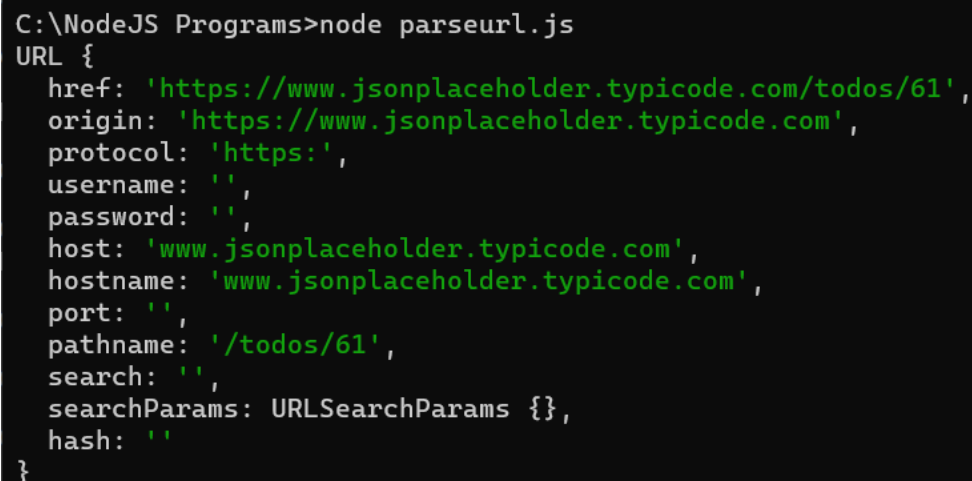
In Code Snippet 7, the `URL` constructor is used to parse the URL string provided. The result of this parsing operation is stored in the `parsedURL` object, which contains various components of the URL, including `protocol`, `hostname`, `port`, and `pathname`.

Subsequently, the contents of the `parsedURL` object are displayed on the console using the line of code, `console.log(parsedUrl)`.

2. Save the file as `parseurl.js`.
3. To parse the given URL string, in the Command Prompt, type:

```
node parseurl.js
```

The command executes and displays the response data from the remote server, as shown in Figure 4.7.



```
C:\NodeJS Programs>node parseurl.js  
URL {  
  href: 'https://www.jsonplaceholder.typicode.com/todos/61',  
  origin: 'https://www.jsonplaceholder.typicode.com',  
  protocol: 'https:',  
  username: '',  
  password: '',  
  host: 'www.jsonplaceholder.typicode.com',  
  hostname: 'www.jsonplaceholder.typicode.com',  
  port: '',  
  pathname: '/todos/61',  
  search: '',  
  searchParams: URLSearchParams {},  
  hash: ''  
}
```

Figure 4.7: Response Data From Remote Server

The output shown in Figure 4.7 can also be obtained if the code is executed using Terminal mode in the VS Code.

4.5.2 Building the URL

The `url.format` method is used to create a URL string from a URL object or URL components.

Consider that the user wants to create a URL string using these URL components:

- `protocol: 'https:',`
- `hostname: 'www.jsonplaceholder.typicode.com',`
- `port: '8080',`
- `pathname: '/todos/61'`

To do so, the user must perform these steps:

1. Type the code given in Code Snippet 8 in the text editor.

Code Snippet 8:

```
const url = require('url');
const urlObject = {
  protocol: 'https:',
  hostname: 'www.jsonplaceholder.typicode.com',
  port: '8080',
  pathname: '/todos/61'
};
const urlString = url.format(urlObject);
console.log(urlString);
```

Code Snippet 8, begins by importing the URL module using the `require` function, assigning it to a constant variable `url`.

Then, it creates an URL object called `urlObject` by specifying various components of the URL, including the `protocol` (`'https:'`), `hostname` (`'www.jsonplaceholder.typicode.com'`), `port` (`'8080'`), and `pathname` (`'/todos/61'`).

The `url.format` method is used to create a complete URL string from the individual components.

Finally, the URL string generated is displayed on the console.

2. Save the file as `buildurl.js`.

3. To build the URL string, in the Command Prompt window, at the Command Prompt, execute the command as:

```
node buildurl.js
```

The command executes and displays the response data from the remote server, as shown in Figure 4.8.

```
C:\NodeJS Programs>node buildurl.js  
https://www.jsonplaceholder.typicode.com:8080/todos/61
```

Figure 4.8: Response Data From Remote Server

4.5.3 Retrieving Specific Components of the URL

Users can also retrieve specific components of the URL using methods, such as:

Hash	• <code>url.hash</code>
Host	• <code>url.host</code>
Host Name	• <code>url.hostname</code>
Href	• <code>url.href</code>
Origin	• <code>url.origin</code>
Password	• <code>url.password</code>
Path Name	• <code>url.pathname</code>
Port	• <code>url.port</code>
Protocol	• <code>url.protocol</code>

For example, consider that the user wants to retrieve the `protocol`, `hostname`, and `pathname` for the specified URL. To do so, the user must perform these steps:

1. Type the code given in Code Snippet 9 in the text editor.

Code Snippet 9:

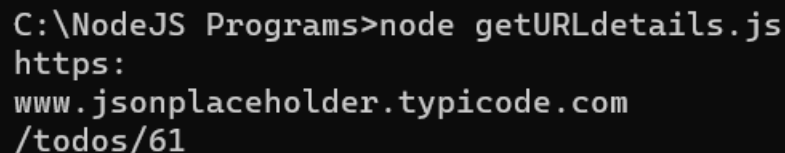
```
const parsedUrl = new URL  
( 'https://www.jsonplaceholder.typicode.com/todos/61' );  
console.log(parsedUrl.protocol);  
console.log(parsedUrl.hostname);  
console.log(parsedUrl.pathname);
```

Code Snippet 9, begins by creating an URL object called `parsedURL` by passing the URL provided to the `URL` constructor. Then, the specific components of the URL are displayed on the console.

2. Save the file as `getURLdetails.js`.
3. To build the URL string, in the Command Prompt window, at the Command Prompt, execute the command as:

```
node getURLdetails.js
```

The command executes and displays the response data from the remote server, as shown in Figure 4.9.



```
C:\NodeJS Programs>node getURLdetails.js  
https:  
www.jsonplaceholder.typicode.com  
/todos/61
```

Figure 4.9: Response Data From Remote Server

Consider the flight reservation system discussed in the `http` module. When developing a flight reservation system, the developers can use the `url` module to:

- Handle incoming requests with encoded data in query strings or paths.
- Build dynamic query strings defining the search criteria for flight booking.
- Parse incoming request URLs to obtain the search criteria for flight booking.
- Generate confirmation pages or booking summary pages with appropriate URLs.
- Redirect users to payment gateways or external services with properly constructed URLs.
- Create links to e-mail confirmations or itinerary details.

4.6 Events Module

The `events` module provides the `EventEmitter` class, which serves as the foundation for working with events in Node.js. Event emitters are objects that can emit named events, allowing servers to listen to those events and respond to them asynchronously. Users can also create custom event emitters by extending the `EventEmitter` class.

The `events` module is an event-driven architecture, which helps users to work with events and event emitters.

This `events` module allows users to:

Build applications where various components, such as objects or modules, communicate by emitting and listening for events.

Register event listeners that respond to I/O events, such as reading a file, receiving an HTTP request, or handling incoming data from a socket.

Handle user interactions, such as button clicks, mouse movements, and keyboard inputs.

Build real-time applications, such as chat systems, multiplayer games, and data streaming applications.

Notify clients of real-time changes or updates.

Create custom events specific to the requirements of the application to represent various states or actions and trigger them in specific conditions.

Table 4.4 shows the methods of the `events` module.

Methods	Description
<code>on(event, listener)</code>	Registers a listener function to be executed whenever the specified event is emitted.
<code>once(event, listener)</code>	Registers a one-time listener function that will be automatically removed after it is invoked the first time.
<code>removeListener(event, listener)</code>	Removes a specific listener from the listener array for the specified event.
<code>setMaxListeners(n)</code>	Sets the maximum number of listeners that can be added for an event. The default is 10.
<code>emit(event[, ...args])</code>	Emits the specified event with an optional list of arguments, triggering the registered listener functions for that event.

Table 4.4: Methods of `events` Module

4.6.1 Creating an Event-Driven Notification System

Consider that a user wants to create a notification system where events are used to trigger notifications when specific conditions are met. To do so, the user must perform these steps:

1. Type the code given in Code Snippet 10 in the text editor.

Code Snippet 10:

```
const EventEmitter = require('events');
class NotifyEmitter extends EventEmitter {}
const notifyEmitter = new NotifyEmitter();
// Register event listener for sending notifications
notifyEmitter.on('sendNotification', (message) => {
  console.log(`Sending notification: ${message}`);
});
// Simulate events triggering notifications
setTimeout(() => {
  notifyEmitter.emit('sendNotification', 'Meeting
Reminder');
}, 2000);
setTimeout(() => {
  notifyEmitter.emit('sendNotification', 'Reminder to
call client');
}, 4000);
setTimeout(() => {
  notifyEmitter.emit('sendNotification', 'Product
Delivery Reminder');
}, 6000);
```

Code Snippet 10 creates a custom event emitter class `NotifyEmitter` by extending the `EventEmitter` class from the `events` module. This class will serve as the foundation for the notification system.

Then, an instance of the `NotifyEmitter` class, named `notifyEmitter`, is created. This instance is used to work with event handling within the application.

The code uses the `.on(eventName, listener)` method to register an event listener. In this case, an event listener is registered for the `sendNotification` event.

This method takes two arguments:

- `eventName`: Specify the `eventName`, such as `sendNotification`.
- `listener`: Define the `listener` function, such as the message that executes when the event is emitted.

When the `sendNotification` event is emitted, the associated listener function executes, receiving a message as an argument and logging the message to the console.

To simulate events triggering notifications, the `setTimeout` function is used. After specific time intervals (in this case, after 2, 4, and 6 seconds), the `.emit(eventName, [args])` method is used to emit the `sendNotification` event. Each time, a different notification message is passed as the second argument.

2. Save the file as `eventemit.js`.
3. To trigger the event `sendNotification`, in the Command Prompt window, at the Command Prompt, execute the command as :

```
node eventemit.js
```

The command executes and simulates a notification system where events trigger notifications based on specific conditions, as shown in Figure 4.10.

```
C:\NodeJS Programs>node eventemit.js
Sending notification: Meeting Reminder
Sending notification: Reminder to call client
Sending notification: Product Delivery Reminder
```

Figure 4.10: Notification System

The `notifyEmitter.emit('sendNotification', message)` line triggers the `sendNotification` event, and the registered listener responds by sending the notification message to the console.

4.6.2 Managing Event Listeners in the `sendNotification` Event

Users can register multiple listeners for the same event. In case of multiple listeners, all of them will be invoked in the order they were registered.

The `.once(eventName, listener)` method registers a listener that is automatically removed after it is invoked once.

To register multiple listeners for the `sendNotification` event using `.once` method, perform these steps:

1. Type the code given in Code Snippet 11 in the text editor.

Code Snippet 11:

```
const EventEmitter = require('events');
class NotifyEmitter extends EventEmitter {}
const notifyEmitter = new NotifyEmitter();
// Register event listener for sending notifications
notifyEmitter.on('sendNotification', (message) => {
  console.log(`Sending notification1: ${message}`);
});
notifyEmitter.on('sendNotification', (message) => {
  console.log(`Sending notification2: ${message}`);
});
notifyEmitter.once('sendNotification', (message) => {
  console.log(`Sending notification-once: ${message}`);
});
notifyEmitter.emit('sendNotification', 'Meeting Reminder');
notifyEmitter.emit('sendNotification', 'Reminder to call client');
notifyEmitter.emit('sendNotification', 'Product Delivery Reminder');
```

2. Save the file as `eventonce.js`.
3. To trigger the `sendNotification` event, in the Command Prompt window, at the Command Prompt, execute the command as:

```
node eventonce.js
```

The command executes and displays the response, as shown in Figure 4.11.

```
C:\NodeJS Programs>node eventonce.js
Sending notification1: Meeting Reminder
Sending notification2: Meeting Reminder
Sending notification-once: Meeting Reminder
Sending notification1: Reminder to call client
Sending notification2: Reminder to call client
Sending notification1: Product Delivery Reminder
Sending notification2: Product Delivery Reminder
```

Figure 4.11: Response Data From Notification System

Figure 4.11 shows that two listeners are registered with `.on` method. These listeners respond to every `sendNotification` event, while the listener registered with the `.once` method responds only to the first `sendNotification` event. After that, this listener is automatically removed.

Consider the flight reservation system discussed in the `http` module. When developing a flight reservation system, the developers can use the `events` module to:

- Listen and handle incoming client requests.
- Update the availability of flights in real-time when a flight booking is in progress.
- Initiate updates related to a flight booking such as seat booking or meal booking.
- Send reminders for upcoming flights.
- Send reminders for Web check-in.

4.7 Summary

- Node.js built-in modules offer essential functionality for a wide range of tasks, including Web servers, file systems, and more, with benefits including performance, reliability, and interoperability.
- The Node.js `http` module simplifies the creation of HTTP servers and requests, providing various methods and classes for interacting with HTTP protocols.
- Common classes in the `http` module in Node.js include `http.Server`, `http.ClientRequest`, and `http.ServerResponse`.
- The Node.js `url` module offers utilities for parsing and manipulating URLs.
- The Node.js `events` module uses the `EventEmitter` class to facilitate event-driven programming for building applications and handling I/O, user interactions, real-time systems, and custom events.

Test Your Knowledge

1. Which method is used to bind and listen on the specified port of the HTTP server?

- a) `http.createServer`
- b) `server.listen`
- c) `server.close`
- d) `request.write`

2. You are working on a Node.js application. You want to send an HTTP request to your remote server including the `request` method, headers, and data to be sent to the server. Which method from the `http.ClientRequest` class will you use?

- a) `request.end([data][, encoding][, callback])`
- b) `response.write(chunk, [encoding], [callback])`
- c) `request.write(chunk[, encoding][, callback])`
- d) `response.writeHead(statusCode, [statusMessage], [headers])`

3. What will be the output of the given code in the browser?

```
const http = require('http');
const server = http.createServer((req, res) => {
  if (req.method === 'GET') {

  } else if (req.method === 'POST') {

  }
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Node.js comes with a variety of built-in
modules');
});

server.listen(3000, () => {
  console.log(`Server is listening on port 3000`);
});
```

- a) It will display `Server is listening on port 3000`
- b) It will display `Node.js comes with a variety of built-in modules`
- c) Error in the code
- d) It will display nothing

In the given code, add the missing statements:

```
const http = require('http');
const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/html') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write('<b>You are working on a
Node.js</b><br/>');
    res.write('<span style="font-weight:Bold;
color:Blue">Now you have started working on
HTTP</span>');
    res.end();
  } else {
    // Add missing statements
  }
});
const port = 3000;
server.listen(port, () => {
  console.log(`Server is listening on port ${port}`);
});
```

- a) `res.writeHead(404, { 'Content-Type': 'text/plain' });`
`res.end('404 Not Found');`
- b) `res.end();`
- c) Nothing to add to this code
- d) `console.log(`Server is listening on port ${port}`);`

4. What is the use of the `url.parse` method in Node.js?

- a) It is used to generate URLs
- b) It is used to parse a URL string into an object
- c) It is used to validate the structure of the URL
- d) It is used to convert an object to a URL

Answers to Test Your Knowledge

1	b
2	c
3	b
4	a
5	b

Try It Yourself

1. Create a simple HTTP server that responds with a plain text message: You have started working on HTTP server.
 - a) Import the `http` module.
 - b) Create an HTTP Server.
 - c) Handle Requests.
 - d) Send a Response You have started working on HTTP server.
 - e) Specify the listening Port as 3000.
 - f) Display the result on the browser.

2. You are working on an application in which you must get the response from the HTTP server in HTML format.
 - a) Import the `http` module.
 - b) Create an HTTP Server.
 - c) Check the HTTP request method and URL of the incoming request.
 - d) Add `if` condition, check if HTTP the request is a `get` method and check if the request URL is equal to `/html`.
 - e) Set status code to 200 and the response from the HTTP server is in plain text.
 - f) Add the response You are working on a Node.js application. Use font color as Blue and font weight as Bold.
 - g) If the requested URL is not the root (`/html`), then display 404 Not found.
 - h) Specify the port as 3000.
 - i) Display the result on the browser.

3. You are working on a chat application and you must create an Event-Driven Notification System.
 - a) Create a custom event emitter class `ChatEmitter`.
 - b) Create an instance of the `ChatEmitter` class, named `chatEmitter`.
 - c) Register an event listener for:
 - Customer joined
 - Customer left
 - New message
 - d) Simulate events triggering notifications using 3, 5, and 7 seconds.
 - e) Display the result on the browser.